CST1340 INFORMATION IN ORGANISATIONS

Coursework 2 – Report

Title: 2-page bookshop website

April intake Semester 2

Date of submission: 09.12.2025

Group members:

#	Student Number	Full Name
1	M01067508	Jasveer Poorunsing
2	M01068406	Hansraj Choytun
3	M01068114	Deevyesh Aarav Ujoodha
4	M01068109	Preetish Pudaruth

Contents

TABLE OF FIGURES 2

Project Overview 3

Section 1 – HTML 4

Key HTML Code Snippets 5

CSS Styling 9

Key CSS Code Snippets 9

Reflection 12

Section 2 – XML 12

Key XML Code Snippets 13

Reflection 14

Section 3 – Validation 15

Key DTD Code Snippets 15

Reflection 17

Section 4 – Transformation 18

Key XSL Code Snippets 19

Reflection 22

Potential Improvements 22

Running Instructions 23

Conclusion 23

References 24

Use of AI Tools in Project Development 24

TABLE OF FIGURES

Figure 1: HTML/CSS design 5
Figure 2: Declaration and Head Section 5
Figure 3: Genres and Authors section 6
Figure 4: Opening times section 7
Figure 5: Author Hover Tooltip 9
Figure 6: Opening Times 10
Figure 7: Smooth Scrolling and Section Highlighting 11
Figure 8: XML book library 13
Figure 9: DTD validation 15
Figure 10: DTD Hierarchy diagram 17
Figure 11: XML/XSLT design 18
Figure 12: XML Declaration 19
Figure 13: XSLT Stylesheet Declaration 19
Figure 14: Output Settings 19
Figure 15: Root Template 20
Figure 16: Root Template 20
Figure 17: Link Template 20
Figure 18: Books Table 21
Figure 19: Closing the Template 21

Project Overview

The objective of this project was to create a 2-page website. The main page uses HTML to properly structure the webpage and CSS for the visual design and responsive layout. The program page uses XSL to create a dynamic book catalogue that displays book information from XML data. It uses XSLT to implement XML-to-HTML conversion and DTD to validate the XML structure.

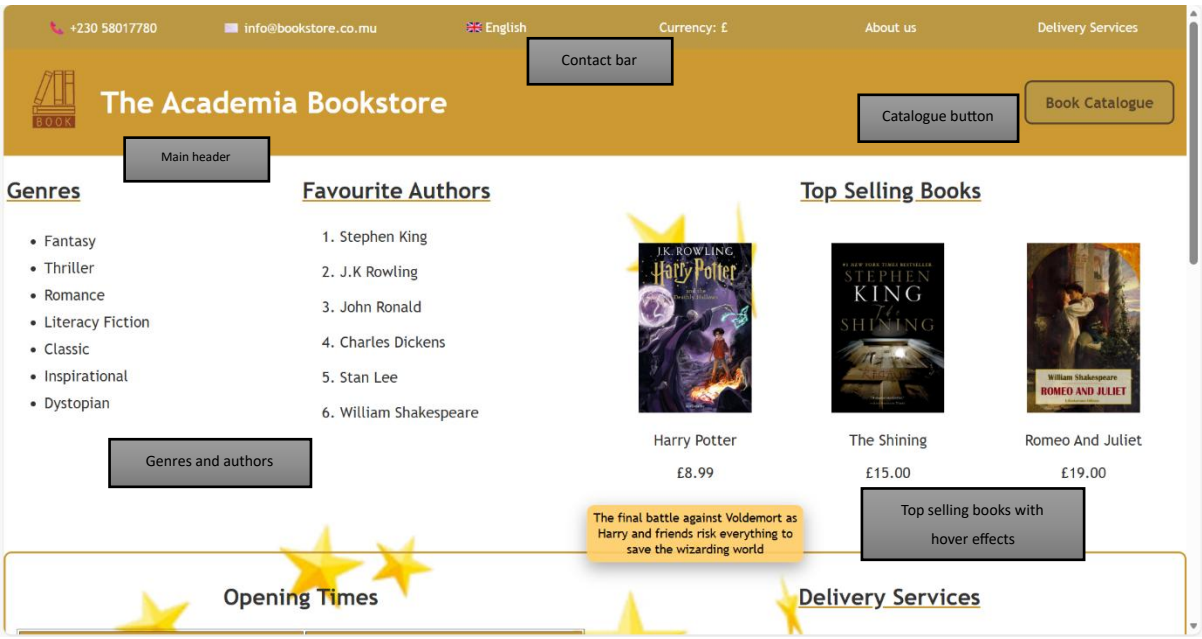
The design features a professional bookstore aesthetic with a consistent gold/brown colour scheme and clean typography. The interface includes a contact bar, logo header, navigation system with dropdown menus, search feature and a book catalogue table.

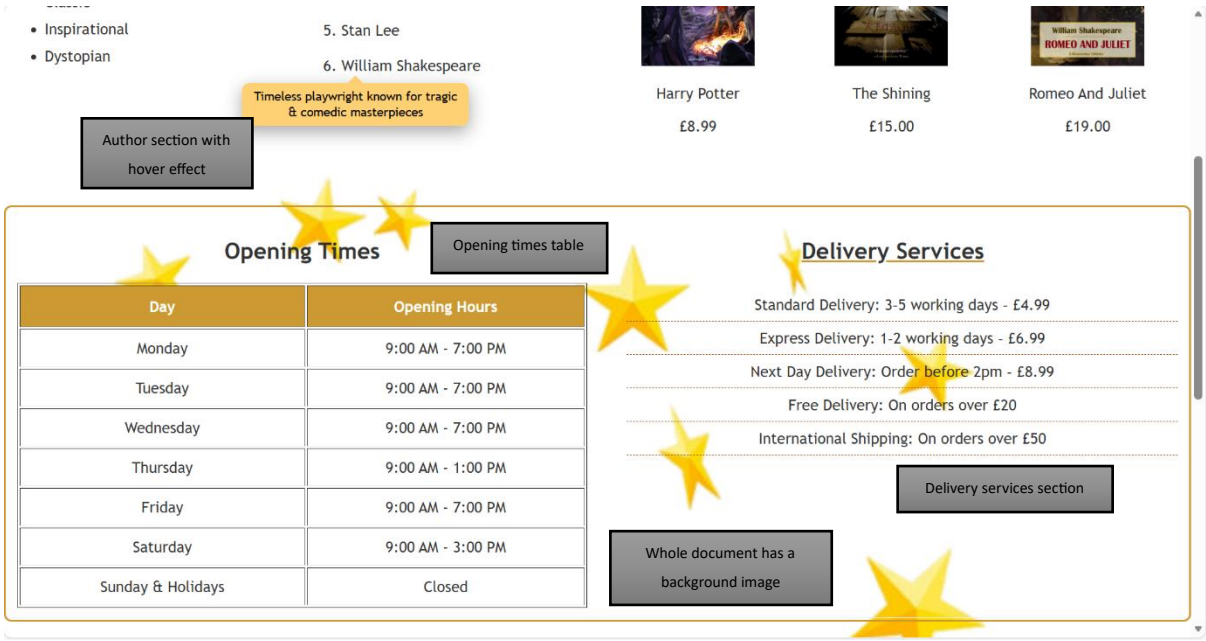
This coursework was completed by our group where each member had a significant contribution. Hansraj designed the HTML page structure, Jasveer styled the HTML page using CSS, Deevyesh created the XML data, the DTD validation and added the hover effects in both pages, while Preetish handled XSLT transformation. All components integrated seamlessly through regular team coordination.

This report consists of a detailed analysis of all the codes. It examines limitations of the current implementation and challenges encountered during the design, highlights key learning outcomes, presents annotated code snippets demonstrating core functionalities and includes clear running instructions for execution of XSLT file.

Section 1 – HTML

The HTML document represents a static website for a bookstore called The Academia Bookstore. The page is designed to present essential business information such as contact details, book categories, top-selling books, opening times, delivery services, and a newsletter subscription form. The layout makes extensive use of HTML tables for structure and CSS for presentation and styling.





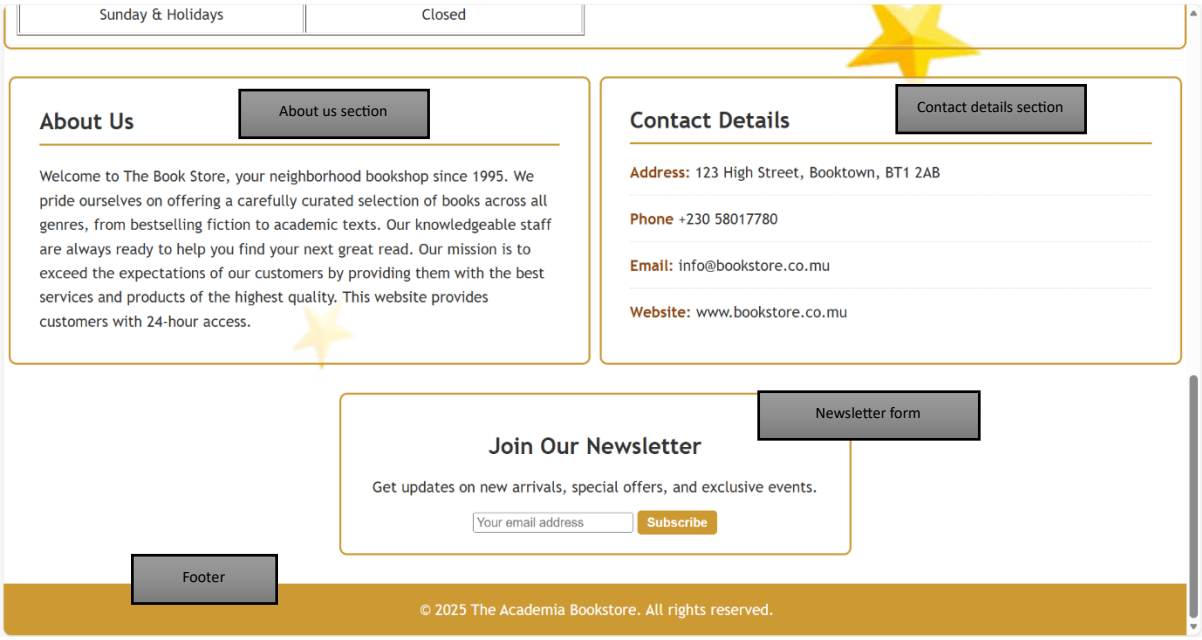


Figure 1: HTML/CSS design

Key HTML Code Snippets

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Bookstore</title>
    <link rel="stylesheet" href="styles.css">
</head>
```

Figure 2: Declaration and Head Section

- <!DOCTYPE html> declaration specifies that the document follows HTML5 standards ensuring compatibility with modern browsers.
- lang="en" attribute improves accessibility and search engine optimisation by indicating that the page content is in English.
- UTF-8 encoding allows special characters (e.g £, emojis) to display correctly.
- The <title> defines the browser tab name.
- The link tag connects the page to an external CSS file, separating content from design and improving maintainability.

```html
<!-- Genres and Authors -->
<table class="two-col-layout"border="0" width="100%">
    <tr valign="top">
        <td width="25%">
            <h2 class="underlined-heading">Genres</h2>
            <ul>
                <li>Fantasy</li>
                <li>Thriller</li>
                <li>Romance</li>
                <li>Literacy Fiction</li>
                <li>Classic</li>
                <li>Inspirational</li>
                <li>Dystopian</li>
            </ul>
        </td>

        <td width="25%">
            <h2 class="underlined-heading">Favourite Authors</h2>
            <ol>
                <li class="author">
                    Stephen King
                    <span class="author-desc">Master of horror & supernatural fiction</span>
                </li><br>

                <li class="author">
                    J.K Rowling
                    <span class="author-desc">British novelist and creator of Harry Potter</span>
                </li><br>

                <li class="author">
                    John Ronald
                    <span class="author-desc">Fantasy author best known for Middle-earth stories</span>
                </li><br>
```

Figure 3: Genres and Authors section

- An unordered list is used for genres, as no ranking is required.
- An ordered list is used for favourite authors, demonstrating popularity.
- Each author includes a description using elements, styled with CSS, allowing hover effects.

```html
<!-- Opening Times & Delivery Services -->
<table class="opening-times-table" border="0" width="100%">
    <tr valign="top">
        <td width="50%">
            <h2>Opening Times</h2>
            <table border="1" width="100%">
                <tr>
                    <th>Day</th>
                    <th>Opening Hours</th>
                </tr>
                <tr>
                    <td align="center">Monday</td>
                    <td align="center">9:00 AM - 7:00 PM</td>
                </tr>
                <tr>
                    <td align="center">Tuesday</td>
                    <td align="center">9:00 AM - 7:00 PM</td>
                </tr>
                <tr>
                    <td align="center">Wednesday</td>
                    <td align="center">9:00 AM - 7:00 PM</td>
                </tr>
                <tr>
                    <td align="center">Thursday</td>
                    <td align="center">9:00 AM - 1:00 PM</td>
                </tr>
                <tr>
                    <td align="center">Friday</td>
                    <td align="center">9:00 AM - 7:00 PM</td>
                </tr>
                <tr>
                    <td align="center">Saturday</td>
                    <td align="center">9:00 AM - 3:00 PM</td>
                </tr>
                <tr>
                    <td align="center">Sunday & Holidays</td>
                    <td align="center">Closed</td>
                </tr>
            </table>
        </td>
```

Figure 4: Opening times section

This section displays Opening hours in a bordered table where the following tags are used:

- <table> - creates the table.
- <th> - creates a header cell (bold and centered by default).
- <tr> - creates a row.
- <td> - creates a data cell.

Furthermore, the HTML file is organized into several key sections:

(Refer to zip file CST1340 project, code: bookshop.html)

1. The **main header** section consists of
    - The bookstore logo (<img>)
    - The website title (<h1>)
    - A call-to-action button linking to the book catalogue webpage.
2. The **top selling books** consists of
    - Cover Images
    - Book Titles
    - Prices
    - Short Descriptions

Books are placed in a grid-style table to display them neatly side by side. The use of CSS classes such as .book, .cover and .desc allows hover effects and visual enhancements through styling.

3. The **delivery services** section displays the Delivery options in a list.
4. The **About Us** section introduces the bookstore's background and mission.
5. The **Contact Details** section presents key contact information using icons for better visual clarity.
6. The **newsletter form** consists of:
    - An email input field
    - A submit button
7. The **footer** displays copyright information and concludes the page in a clean, professional way.

CSS Styling

CSS provides the visual design and responsive behaviour of the website. Key syling includes the colour scheme, layout structure, and interactive elements like hover effects.

Key CSS Code Snippets

```
.author .author-desc {
    position: absolute;
    left: 50%;
    transform: translateX(-200%) translateY(10px); /* Initial offser for hidden state */
    bottom: -55px; /* Position tooltip below author element */
    width: 220px;
    max-width: 90vw; /* Ensures tootip fits within viewport on smaller screen */
    background: #FFCC66E6;
    color: #000;
    padding: 6px 10px;
    border-radius: 8px;
    box-shadow: 0 6px 18px #00000059;
    font-size: 13px;
    text-align: center;
    line-height: 1.3;
    opacity: 0;
    visibility: hidden; /* Prevents interaction when hidden */
    pointer-events: none; /* Disables mouse events when hidden */
    z-index: 15; /* Ensures tooltip appears above other elements */
    transition: opacity 0.25s ease, transform 0.25s ease; /* Smooth slide animation */
}

.author:hover .author-desc {
    opacity: 1; /* Makes tooltip visible on hover */
    visibility: visible;
    transform: translateX(-100%) translateY(0); /* Moves tooltip into view */
}

.author .author-desc::after {
    content: ""; /* Creates a pseudo-element for tooltip arrow */
    position: absolute;
    left: 50%;
    bottom: 100%;
    transform: translateX(-50%);
    border-left: 8px solid transparent;
    border-right: 8px solid transparent;
    border-bottom: 8px solid #FFCC66E6;
}
```

Figure 5: Author Hover Tooltip

The CSS block defines a hover effect for author names, it works the same as the book cover hover effect in the as when you hover over author, a small description box appears below it with a smooth animation. The .author .author-desc, defines a creates a hidden info box that shows up when you hover over an author's name. It defines the tooltip's appearance, positioning, size, text, styling and animation. The hover behaviour comes entirely from CSS selectors like .author: hover .author-desc. The HTML just simply provides the structure.

```
/* OPENING TIMES TABLE */
.opening-times-table {
    border: 2px solid #CC9933;
    border-radius: 8px;  /* Rounded corners */
    overflow: hidden;
    margin: 10px 0; /* Vertical spacing */
    width: 100%;
}

.opening-times-table th {
    background: #CC9933;
    color: white;
    padding: 12px;
    text-align: center;
}

.opening-times-table td {
    padding: 10px;
    border-bottom: 1px solid #eee;
    text-align: center;
}
```

Figure 6: Opening Times

- Rounded corners soften the design.
- Borders clearly separate opening times data.

```
/* Smooth scrolling */
html {
    scroll-behavior: smooth;
}

/* Section highlighting */
.about-us,
.delivery-services {
    scroll-margin-top: 20px;
}

.about-us:target,
.delivery-services:target {
    animation: highlight 2s ease;
}

@keyframes highlight {
    0%, 100% { background-color: transparent; }
    50% { background-color: rgba(255, 204, 102, 0.3); }
}
```

Figure 7: Smooth Scrolling and Section Highlighting

- scroll-behavior: smooth makes page scrolling gradual instead of instant
- scroll-margin-top adds space above sections when navigating to them
- :target applies styles when a section is reached via an anchor link
- @keyframes highlight animates a temporary background colour effect

These effects:

- Improve navigation experience when clicking internal links.
- Highlight the selected section briefly.
- Help users identify where they have navigated to.

There are additional CSS styles applied to different parts of the HTML file. The stylesheet is clearly commented so that it is easy to understand the CSS styles.

(Refer to zip file CST1340 project, code: styles.css)

Reflection

The website demonstrates several strengths in its design and structure. The content is clearly organised into well-defined sections, making navigation intuitive and user-friendly. Headings and lists are used consistently, which improves readability and helps users quickly locate information. The use of an external CSS stylesheet enhances maintainability by separating content from presentation. Hover effects, smooth scrolling, and section highlights are implemented using only CSS, improving usability without relying on JavaScript. In addition, the clear separation between HTML structure and CSS presentation follows good web development practice.

Despite these strengths, the webpage also has some limitations. Tables are used for layout purposes rather than purely for tabular data, which goes against modern web design practices. In addition, some inline styling is present, such as margin adjustments, reducing consistency and maintainability. The website is largely static and does not use JavaScript to enhance interactivity. Furthermore, the design has limited responsiveness, meaning it may not display optimally on smaller screens or mobile devices.

Section 2 – XML

This XML defines a structured catalogue of books for a bookshop, with validation via a DTD and display transformation via an XSL stylesheet.

Key XML Code Snippets

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE bookshop SYSTEM "program.dtd">
<?xml-stylesheet type="text/xsl" href="program.xsl"?>

<bookshop>

    <book>
        <title>The Great Gatsby</title>
        <author>F. Scott Fitzgerald</author>
        <genre>Classic</genre>
        <description>A tragic story of wealth, obsession, and the American Dream set in the Roaring Twenties.</description>
        <price>9.99</price>
    </book>

    <book>
        <title>1984</title>
        <author>George Orwell</author>
        <genre>Dystopian</genre>
        <description>A chilling depiction of totalitarianism, surveillance, and the loss of individual freedom.</description>
        <price>8.50</price>
    </book>
```

```xml
    <book>
        <title>Romeo and Juliet</title>
        <author>William Shakespeare</author>
        <genre>Tragedy</genre>
        <description>A timeless tragic romance about two young lovers whose families' feud leads to heartbreak and fate-d
        <price>19.00</price>
    </book>

    <link href="bookshop.html">Return to Main Page</link>
</bookshop>
```

Figure 8: XML book library

- The program first declares the XML version and character encoding so parsers know how to read the file.
- “encoding="UTF-8" specifies how text characters are stored in the file. This ensures that special characters are read correctly by the parser.
- The DOCTYPE with DTD points to an external DTD named program.dtd. If present and loaded, the DTD enforces structure (e.g., which elements are allowed, their order, and whether required fields must exist).
- <?xml-stylesheet type="text/xsl" href="program.xsl"?> associates the XML with an XSL stylesheet (program.xsl) that can transform the raw data into formatted HTML (or other output) for display.
- <bookshop> is the root container holding the entire catalogue. All content must be inside it.

- Each <book> is a single record with consistent child elements such a genre, price, description, author and title for each book. It is repeated for each book making the XML a structured catalogue.
- <link href="bookstore3.html">Return to Main Page</link> is a navigation marker that points to another page. It contains both the destination (href) and the visible text ("Return to Main Page"). Through XSL, this custom XML tag is transformed into a standard HTML <a> link so users can click it.

Reflection

XML has taught me how to structure, store, and transport data in a format that's accessible to both humans and machines. It's not a programming language but a markup language that emphasizes on structure, clarity and compatibility, between systems. I learnt how information can be arranged hierarchically using nested tags. It teaches us to think of data as a tree diagram where each piece of information is wrapped in tags that define its role.

XML can be challenging to beginners. Validation in XML is crucial as it ensures that documents are not only well-formed but also follow strict structural rules. This is achieved using DTD and XML Schema. Moreover, if we are not careful, small mistakes like missing closing tags or incorrect nesting can break the entire document.

To overcome these challenges, we practiced writing small XML snippets and validating them often to become more comfortable with the topic. We also experimented with XML editors and validation tools that showed the errors We made in real time. This process helped us better understand how xml works.

Section 3 – Validation

The Document Type Definition (DTD) defines the structure and rules of an XML document. It specifies:

Which elements exist

- How they are nested
- What kind of data they contain
- Which attributes are required or optional
- It describes a simple XML schema for a bookshop catalogue.

Key DTD Code Snippets

```
<!ELEMENT bookshop (book+, link)>
<!ELEMENT book (title, author, genre, description, price)>
<!ELEMENT title (#PCDATA)>
<!ELEMENT author (#PCDATA)>
<!ELEMENT genre (#PCDATA)>
<!ELEMENT description (#PCDATA)>
<!ELEMENT price (#PCDATA)>
<!ELEMENT link (a)>
<!ELEMENT a (#PCDATA)>
<!ATTLIST a href CDATA #REQUIRED>
```

Figure 9: DTD validation

The first line <!ELEMENT bookshop (book+, link)> declares the root element called bookshop.

- It specifies that inside a bookshop there must be one or more book elements followed by a single link element.
- The plus sign after book means at least one book is required, and the order matters because the sequence is defined as books first and then the link.
- The order is strict: books must appear first, then the link.

<!ELEMENT book (title, author, genre, description, price)>

- Defines the structure of each book element.

- A book must contain exactly one title, one author, one genre, one description and one price.
- These child elements must appear in the specified order.

The line <!ELEMENT title (#PCDATA)> declares that title holds parsed character data: typically contains the name of the book.

<!ELEMENT author (#PCDATA)> declares that **author** holds text data and it contains the author's name.

Similarly, for genre, description and price, #PCDATA was used.

<!ELEMENT link (a)> defines the **link** element, provides a container for hyperlinks in your XML structure, similar to how HTML uses <link> or <a> tags. It must contain exactly one a element.

<!ELEMENT a (#PCDATA)> declares that **a** holds text data which is parsed character data (PCDATA) and it represents the clickable link text.

<!ATTLIST a href CDATA #REQUIRED> declares that the a element must have an attribute called **href**. The attribute is of type CDATA, meaning it can contain any character data (commonly a URL). The keyword #REQUIRED makes this attribute mandatory for every **a** element. It ensures that every link has a valid destination. Without href, the link would be meaningless.

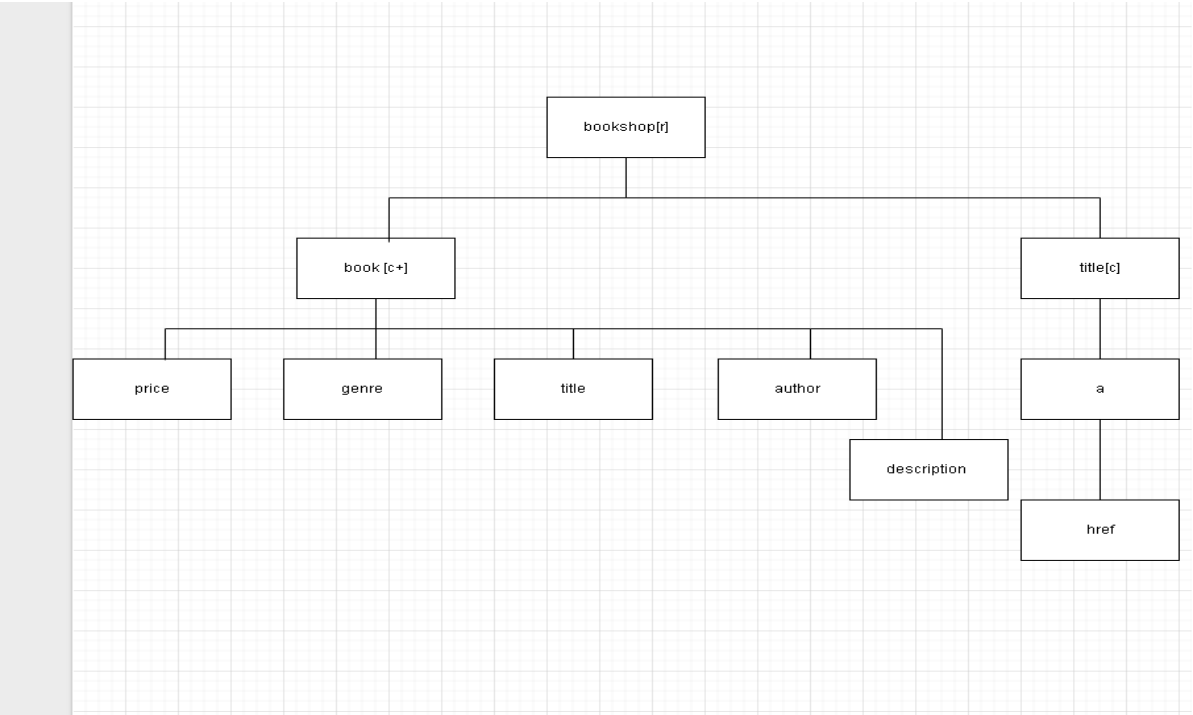


Figure 10: DTD Hierarchy diagram

The tree diagram style breakdown of the DTD that visualize how the elements nest inside each other:

At the top is the root element <bookshop>, contain One or more <book> elements which include title price genre and author and one <link> element which contains an <a> element, The <a> element includes text and must have a required href attribute for the URL.

Reflection

Using DTD has taught us about the rules and structure of an XML document. DTD defines which elements can exist, how they are ordered in a document, what attributes they can have, and what type of data they contain. It acts as a blueprint that guarantees XML documents are valid. DTD has shown me how elements are organized, which ones are required, and in what order they must appear. DTD explains whether an element contains text (PCDATA), other elements, or a mix of both.

DTD can be challenging as we need to understand strict ordering in content models. For example, the line <!ELEMENT book (title, author, genre, price)> enforces that these child elements must appear in the exact sequence given and if you swap the order, the XML becomes invalid.

Section 4 – Transformation

XSLT transforms XML data into HTML, generating content for the book catalogue. Key transformations include data iteration, element mapping and template application.

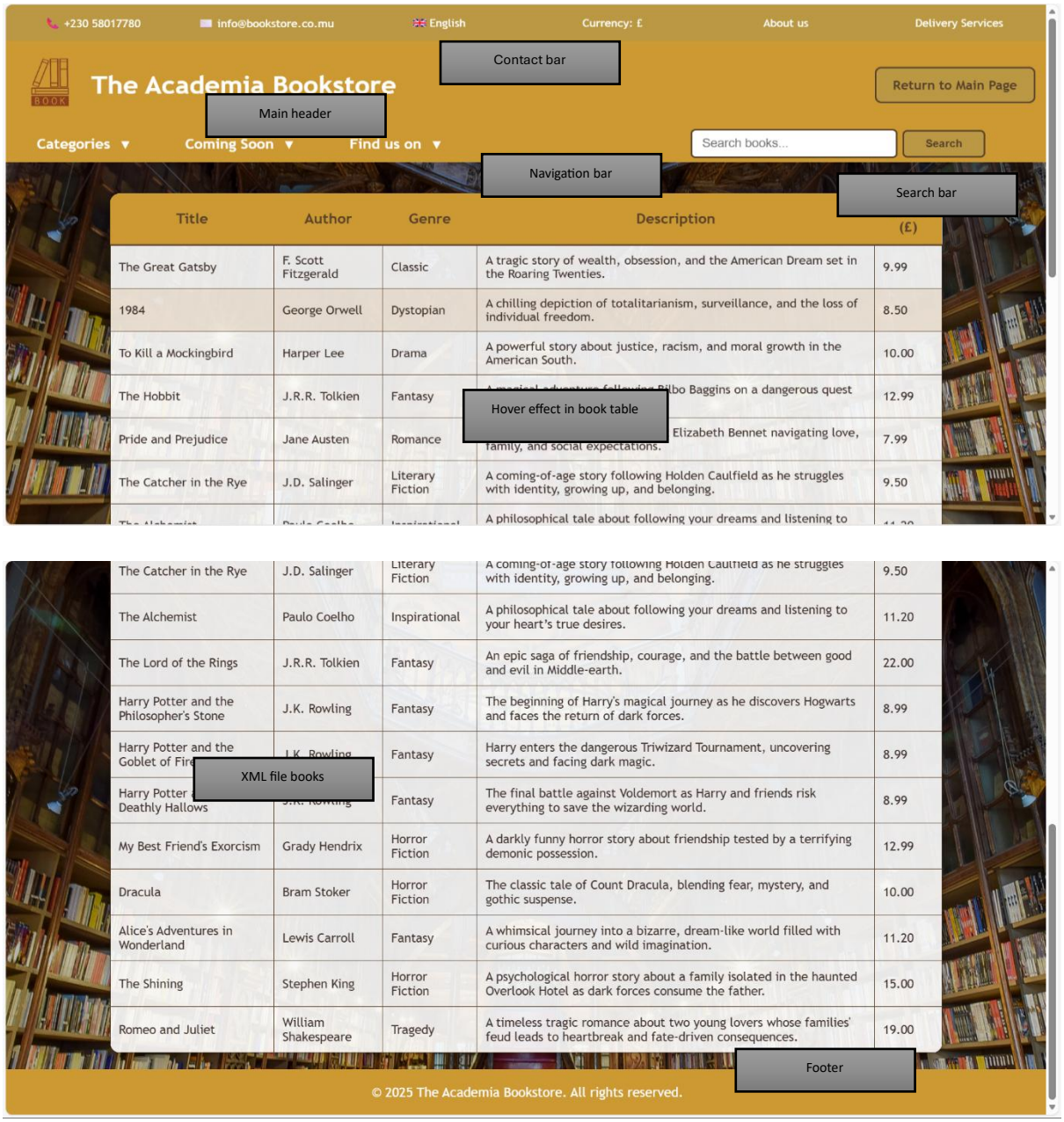


Figure 11: XML/XSLT design

Key XSL Code Snippets

```
<?xml version="1.0" encoding="UTF-8"?>
```

Figure 12: XML Declaration

This states that the file is XML version 1.0 and uses UTF-8 character encoding .

```
<xsl:stylesheet version="1.0"
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
```

Figure 13: XSLT Stylesheet Declaration

What this does:

- Declares that the document is an XSLT stylesheet.
- version="1.0": Uses XSLT 1.0 syntax and functions.
- xmlns:xsl="http://www.w3.org/1999/XSL/Transform": This namespace tells the processor which tags belong to XSLT instructions.

```
<xsl:output method="html" indent="yes" encoding="UTF-8" />
```

Figure 14: Output Settings

Purpose:

- method="html" → Produces an HTML document after transformation.
- indent="yes" → Format the output with indentation for readability.
- encoding="UTF-8" → Ensures the HTML output is UTF-8 encoded.

```
<xsl:template match="/">
```

Figure 15: Root Template

Meaning:

- Matches the root of the input XML.
- Everything inside this template forms the structure of the HTML website.

```
body {
    margin: 0;
    background-image: url('xml_background.jpg');
    background-size: cover;
    background-attachment: fixed;
    background-repeat: no-repeat;
    background-position: center;
    position: relative;
    overflow-x: hidden;
}
```

Figure 16: Root Template

Creates a full-screen, fixed background image.

```
<!--LINK TEMPLATE: Processes and matches link elements from XML -->
<xsl:template match="link">
    <!-- Anchor tag with href from XML attribute and text from element content -->
    <a class="return-button" href="{@href}">
        <xsl:value-of select="."/>
    </a>
</xsl:template>
```

Figure 17: Link Template

Example XML:

```
<link href="bookshop.html">Return</link>
```

Output:

```
<a class="return-button" href="bookshop.html">Return</a>
```

What it does:

- Matches <link> elements in the XML input.
- Creates a styled button (return-button class).
- Uses attribute value {@href} as the link destination.
- Displays the text inside the <link> element.

This starts a loop through all <book> elements in the XML file.

```
<xsl:for-each select="bookshop/book">
```

For each book, the following HTML row is created:

```
<tr>
  <td>[TITLE]</td>
  <td>[AUTHOR]</td>
  <td>[GENRE]</td>
  <td>[DESCRIPTION]</td>
  <td>[PRICE]</td>
</tr>
```

XSLT fills the values using:

```
<xsl:value-of select="title"/>
```

Figure 18: Books Table

The entire HTML structure ends with:

```
</xsl:template>
```

Figure 19: Closing the Template

The xsl file also contains much more styling information about main header, navigation bar, search bar and return button.
(Refer to zip file CST1340 project, code: program.xsl)

Reflection

Working on this XSLT and HTML transformation project has given us a deeper understanding of how XML data can be dynamically converted into a structured, styled web page. We learned how XSLT templates operate, especially how the root template controls the entire output and how looping mechanisms like <xsl:for-each> extract and display XML content. We also gained a clearer appreciation of how CSS can be embedded directly inside an XSLT file to style the final HTML output, allowing XML data and visual presentation to work together seamlessly.

One of the main challenges we faced was maintaining clean separation between XSLT logic and HTML layout. Because both use tag-based syntax, it was easy to misplace brackets or confuse XSL instructions with HTML elements. Ensuring that every template matched the correct part of the XML structure also required careful attention.

Potential Improvements

While complete in its current form, the project could be improved with:

- JavaScript integration for enhanced interactivity.
- Backend connectivity for real-time updates.
- Responsive design improvements for mobile devices.
- Filtering and sorting capabilities for the book catalogue.

Running Instructions

Steps to Run the XSLT file:

1. open the command console on Windows (cmd) or a terminal on Mac.
2. cd < path to your files >
3. and execute the following command: py -m http.server
4. You should see that the server is running if it outputs the following message: Serving HTTP on 0.0.0.0 port 8000 ...
5. http :// localhost :8000/ yourfile.xml

Conclusion

Through this project we deepened our understanding in webpage development It has shown us how HTML, CSS, XML, XSL, and DTD each play a distinct role i.e.

- HTML expertise in creating semantic, accessible page structures
- CSS is used for implementing responsive designs with great visuals.
- XML, DTD and XSLT produce a data-driven websites that are maintainable, scalable and visually appealing.

Effective teamwork such as dividing tasks, peer-reviewing and maintaining effective communication helped us overcome challenges and complete the project effectively.

Overall, this experience enhanced both our technical skills and collaboration abilities, resulting in both educational value and a functional, aesthetically pleasing final product.

References

- **Learn-HTML.org.** *Learn HTML*. Available at: https://www.learn-html.org/ (Accessed: 29 November 2025).
- **Mozilla Developer Network.** *HTML: HyperText Markup Language*. Available at: https://developer.mozilla.org/en-US/docs/Web/HTML (Accessed: 02 December 2025).
- **Imran Siddiq - Web Squadron.** Tooltips on Hover - HTML + CSS. YouTube. Available at: https://youtu.be/mU4a_Iqa4sE?si=SeZdd4BQJnQ0w8fk (Accessed: 03 December 2025).
- **W3Schools.** *XSL Tutorial* — Introduction to XSL. Available at: https://www.w3schools.com/xml/xsl_intro.asp (Accessed: 05 December 2025).

Use of AI Tools in Project Development

I made limited but targeted use of AI tools, specifically **ChatGPT** to assist in specific parts of this project. AI support was applied for debugging CSS styling issues and clarifying XSLT transformation logic. All AI-generated content was carefully reviewed, tested, and adapted to suit the project requirements and compliance with academic integrity standards. The final implementation remains our original work, with AI serving only as a supplementary resource.